# Ledger Reference

*PolyglotPoisoning_CL — shared across every challenge in this category*

Shared across every challenge in this category. Keep it open in a second tab. Workshop: **Polyglot Poisoning: Hands-On Hunting for Linguistic Attacks in AI** — Sahara Al-Madi, Linguistic Security Institute.

> **No lab?** Every exercise in this reference also works against `data/practice_ledger.jsonl` in the repo — the same seven columns, 22 synthetic rows, no network required.

## Connect

1. **Sentinel and Azure access is provided for this workshop.** Refer to the DEATHCon setup instructions for Sentinel, Azure and your `@deathlab.net` account — before the session, not during it.
2. Open the **deathcon2026-sentinel** workspace → Logs blade.
3. Every participant has Reader access. Queries are KQL.

Start here, to see what you are working with:

```
PolyglotPoisoning_CL
| take 20
```

## The intake ledger

Every AI submission to this workspace is logged as one row in one custom table: `PolyglotPoisoning_CL`.

| COLUMN | MEANING |
|---|---|
| `TimeGenerated` | when it arrived |
| `UserPrincipal` | who sent it |
| `Language` | which tongue, at dialect level |
| `Variant` | how the request was shaped |
| `RiskScore` | scored risk — a real number; the wire trips at 5 |
| `Patterns` | detections it tripped — a CSV string, several per row |
| `PromptPreview` | first 200 characters of the request |

## Language codes

You do not need to speak any of these. You need to read the labels.

| | |
|---|---|
| `ar-MSA` | Modern Standard Arabic — the formal written register |
| `ar-EG` | Egyptian Arabic |
| `ar-TN` | Tunisian Arabic |
| `es-ES` | Spanish |
| `fr-FR` | French |
| `de-DE` | German |
| `it-IT` | Italian |
| `pt-PT` | Portuguese |
| `bn-IN` | Bengali |
| `hi-IN` | Hindi |
| `sw-KE` | Swahili |
| `th-TH` | Thai |
| `en-US` | English |

Codes carry region suffixes in this ledger — filter on the full code, e.g. `Language == "ar-TN"`, or use `startswith` for a family.

> **Arabic is not one language in this ledger.** Egyptian and Tunisian differ about as much as varieties of Spanish — different words, different grammar. A filter that handles one may not handle the other at all, which is exactly why they are separate codes.

## Variant — how the request was shaped

| CODE | MEANING |
|---|---|
| V1 | asked plainly — a standard, direct request |
| V2 | asked in a regional variety or dialect |

**Only those two codes ever appear in `Variant`.** Everything else about how a request was shaped — the register it borrowed, the framing it wore, the script it was typed in — is recorded in `Patterns`, not here. Query the codes, not the glosses: `Variant == "V2"`, not `"dialect"`.

**Language says WHERE a request was asked. Variant says WHAT was done to it.** They vary independently — any dialect can carry any handling method — so they are two columns, not one. That independence is the point: a crew that changes both at once is not four incidents. It is one campaign wearing four coats.

Register and framing are not dialects. A request that is only framed — an old manual, a museum label, a novel — stays V1, and the shaping shows up in `Patterns`.

## Patterns — what the detections flagged

Deliberately not listed here. Finding out what lives in this column, and which values are rare, is part of the work.

One thing you do get for free: `Patterns` packs several values into a single comma-separated string. Counting the column directly counts **combinations**, not individual detections. **Split before you count.**

```
PolyglotPoisoning_CL
| where UserPrincipal !endswith "@lb"
| mv-expand stamp = split(Patterns, ",") to typeof(string)
| summarize rows = count() by stamp
| sort by rows asc
```

## Why this table looks like this

At **detection** time, the language is where the attack hides — a filter trained primarily in English may not generalize reliably when the same request arrives in Tunisian or another language variety. At **hunt** time you cannot read every language, so you hunt the labels the detection left behind instead.

That only works if the telemetry carries dialect and technique as columns. Most telemetry does not. This ledger does, which is the whole reason these hunts are possible.

## Take this back to work

These seven columns are not CTF scaffolding. They are close to the minimum an AI gateway should log if you ever want to hunt it: who asked, in which dialect, shaped how, scored what, flagged by which detections, and a preview you can pivot on.

In a 48-prompt pilot against two public safety classifiers, both caught every English attempt and missed a third of the same requests in Egyptian Arabic — while wrongly refusing 60% of ordinary transliterated requests that English never had a problem with. Attacks got through *and* real users got blocked, at the same time, in the same language. Neither is visible in telemetry that logs only `blocked: true`.

Linguistic Firewall, the open-source probe behind this workshop: `github.com/LinguisticSecurityInstitute/linguistic-firewall`
Method, the Report Card format and the full market scan: `almadisahara.com` · `linguisticsecurityinstitute@gmail.com`

> **If your logs cannot answer "which dialect is failing", these are hunts you cannot run at home. That is the gap.**

Flag format for this CTF: `flag{...}`

## License

**© 2026 Sahara Al-Madi, Linguistic Security Institute.** Released under Creative Commons Attribution-NonCommercial 4.0 — use it, teach from it, fork it, adapt it for your own team. Full license and attribution terms are in the **Participant Instructions**, under *License & attribution*. Commercial use: `linguisticsecurityinstitute@gmail.com`